



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий
Кафедра Вычислительной Техники (ВТ)

ОТЧЁТ

по дисциплине

«Разработка программно-аппаратного обеспечения информационных и автоматизированных систем»

Выполнил студент группы
ИКМО-03-25

Фомичев Р.А.

Принял старший преподаватель кафедры ВТ

Данилович Е.С.

Практическая работа выполнена

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва 2025

Оглавление

1. Цель работы	3
2. Описание регистровой модели процессора	4
3. Набор команд.....	5
4. Архитектура программной модели	6
5. Протокол испытаний модели процессора.....	7
Вывод.....	8
Приложение	9

1. Цель работы

Целью данной работы является разработка программной модели процессора архитектуры фон Неймана с безадресной системой команд, а также реализация и отладка ассемблерной программы для вычисления суммы элементов массива в оперативной памяти.

2. Описание регистровой модели процессора

В разработанной модели процессора используются следующие регистры и служебные структуры:

1. Счётчик команд (PC, Program Counter)

Регистр PC содержит адрес текущей выполняемой команды в оперативной памяти. После выборки команды автоматически увеличивается на 1, либо изменяется командами условного и безусловного перехода (JMP, JZ).

2. Регистр инструкции (IR, Instruction Register)

Регистр IR хранит текущую выполняемую инструкцию, считанную из оперативной памяти по адресу PC. Используется для декодирования операции и её аргумента.

3. Стек (STACK)

Стек используется как основной механизм передачи операндов в безадресной архитектуре. Все арифметические и логические операции (ADD, SUB, MUL, CMP) работают исключительно со стеком.

4. Регистр флагов (FLAGS)

Используется один флаг:

Z (Zero Flag) — устанавливается в 1, если результат последней операции равен нулю, иначе 0. Флаг применяется в условном переходе JZ.

5. Оперативная память (RAM)

Оперативная память хранит:

- машинный код программы;
- данные (массив, переменные);
- промежуточные значения.

3. Набор команд

1. PUSH

Команда занесения значения в стек.

Формат: PUSH <значение>

2. LOAD

Команда загрузки значения из памяти в стек.

Формат: LOAD <адрес>

3. STORE

Команда записи значения из стека в память.

Формат: STORE <адрес>

4. ADD

Команда сложения двух целых чисел.

Формат: ADD

5. SUB

Команда вычитания двух целых чисел.

Формат: SUB

6. MUL

Команда умножения двух целых чисел.

Формат: MUL

7. CMP

Команда сравнения двух целых чисел.

Формат: CMP

8. JMP

Команда установки счётчика команд PC на указанный адрес.

Формат: JMP <адрес>

9. JZ

Команда условного перехода при.

Формат: JZ <адрес>

10. HALT

Команда остановки.

Формат: HALT

4. Архитектура программной модели

1. Основные переменные и структуры

- CPU — класс, реализующий модель процессора
- RAM — список, представляющий оперативную память
- STACK — стек операндов
- FLAGS — словарь флагов
- PC, IR — регистры процессора

2. Основные функции

reset() – инициализация процессора, очистка памяти, стека и регистров.

step() – выполнение одного машинного такта:

1. выборка инструкции;
2. декодирование;
3. выполнение операции;
4. обновление флагов и PC.

assemble(program) – двухпроходный ассемблер:

- первый проход — определение меток;
- второй проход — формирование машинного кода.

load_program() – загрузка программы и исходных данных (массива и его длины) в оперативную память.

3. Ассемблерная программа

Программа вычисляет сумму элементов массива, используя цикл и косвенную адресацию.

Используются переменные:

- 190 — сумма
- 191 — индекс
- 199 — длина массива
- 200 — начало массива

5. Протокол испытаний модели процессора

Исходные данные

В оперативную память загружен массив: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], длина массива: 10.

Исходные данные представлены на рисунке 1.

```
def load_program():
    code = assemble(program)
    cpu.RAM[:len(code)] = code

    cpu.RAM[200:210] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    cpu.RAM[199] = 10
```

Рисунок 1 – Исходные данные

Ход выполнения

1. Инициализация суммы и индекса нулём.
2. Последовательный обход массива в цикле.
3. Получение элемента массива через косвенную адресацию.
4. Прибавление элемента к текущей сумме.
5. Увеличение индекса.
6. Завершение цикла при достижении конца массива.

Результат

После выполнения программы:

- процессор корректно завершает работу (HALT);
- в стеке содержится итоговая сумма элементов массива: 55

Результат соответствует ожидаемому значению. Результат представлен на рисунке 2.

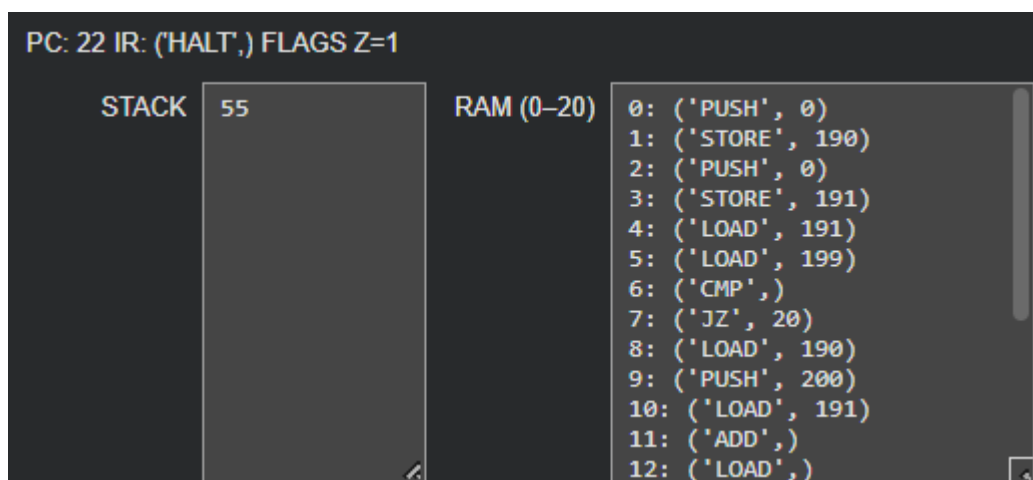


Рисунок 2 – Результат работы программы

Вывод

В ходе выполнения лабораторной работы была разработана программная модель процессора архитектуры фон Неймана с безадресной системой команд. Реализованы основные регистры, стековая организация вычислений, флаговая логика и механизм переходов. Создан ассемблер и успешно написана программа для вычисления суммы элементов массива.

В результате работы:

- реализована корректная модель процессора;
- освоены принципы безадресной архитектуры;
- подтверждена работоспособность модели экспериментально;
- достигнута поставленная цель работы.

Приложение

Листинг 1 – Класс CPU

```
class CPU:
    def __init__(self, ram_size=512):
        self.ram_size = ram_size
        self.reset()

    def reset(self):
        self.RAM = [0] * self.ram_size
        self.PC = 0
        self.IR = None
        self.STACK = []
        self.FLAGS = {"Z": 0}
        self.running = True

    def step(self):
        if not self.running:
            return

        self.IR = self.RAM[self.PC]
        self.PC += 1

        opcode = self.IR[0]
        arg = self.IR[1] if len(self.IR) > 1 else None

        if opcode == "PUSH":
            self.STACK.append(arg)

        elif opcode == "LOAD":
            if arg is None:
                addr = self.STACK.pop()
                self.STACK.append(self.RAM[addr])
            else:
                self.STACK.append(self.RAM[arg])

        elif opcode == "STORE":
```

Продолжение листинга 1

```
self.RAM[arg] = self.STACK.pop()

elif opcode == "ADD":
    b = self.STACK.pop()
    a = self.STACK.pop()
    res = a + b
    self.STACK.append(res)
    self.FLAGS["Z"] = int(res == 0)

elif opcode == "MUL":
    b = self.STACK.pop()
    a = self.STACK.pop()
    res = a * b
    self.STACK.append(res)
    self.FLAGS["Z"] = int(res == 0)

elif opcode == "SUB":
    b = self.STACK.pop()
    a = self.STACK.pop()
    res = a - b
    self.STACK.append(res)
    self.FLAGS["Z"] = int(res == 0)

elif opcode == "JMP":
    self.PC = arg

elif opcode == "JZ":
    if self.FLAGS["Z"] == 1:
        self.PC = arg

elif opcode == "CMP":
    b = self.STACK.pop()
    a = self.STACK.pop()
    self.FLAGS["Z"] = int(a == b)
elif opcode == "HALT":
    self.running = False
```

Листинг 2 – Код функции ассемблера

```
def assemble(program):
    labels = {}
    code = []
    pc = 0
    for line in program:
        if ":" in line:
            labels[line.replace(":", "")] = pc
        else:
            pc += 1
    for line in program:
        if ":" in line:
            continue
        parts = line.split()
        if len(parts) == 1:
            code.append((parts[0],))
        else:
            arg = parts[1]
            arg = int(arg) if arg.lstrip("-").isdigit() else labels[arg]
            code.append((parts[0], arg))

    return code
```

Листинг 3 – Код графического отображения

```
pc_label = widgets.Label()
ir_label = widgets.Label()
flags_label = widgets.Label()

stack_box = widgets.Textarea(
    description="STACK",
    layout=widgets.Layout(width="200px", height="200px")
)

ram_box = widgets.Textarea(
    description="RAM (0–20)",
    layout=widgets.Layout(width="300px", height="200px")
)
```

Продолжение листинга 3

```
def update_gui():
    pc_label.value = f"PC: {cpu.PC}"
    ir_label.value = f"IR: {cpu.IR}"
    flags_label.value = f"FLAGS Z={cpu.FLAGS['Z']}"
    stack_box.value = "\n".join(map(str, reversed(cpu.STACK)))
    ram_box.value = "\n".join(f"{i}: {cpu.RAM[i]}" for i in range(20))

def step_clicked(b):
    cpu.step()
    update_gui()

def run_clicked(b):
    while cpu.running:
        cpu.step()
    update_gui()

def reset_clicked(b):
    cpu.reset()
    load_program()
    update_gui()

step_btn = widgets.Button(description="STEP")
run_btn = widgets.Button(description="RUN")
reset_btn = widgets.Button(description="RESET")

step_btn.on_click(step_clicked)
run_btn.on_click(run_clicked)
reset_btn.on_click(reset_clicked)

display(widgets.VBox([
    widgets.HBox([pc_label, ir_label, flags_label]),
    widgets.HBox([stack_box, ram_box]),
    widgets.HBox([step_btn, run_btn, reset_btn])
]))
```

Листинг 4 – Код программы ассемблера

```
program = [  
    "PUSH 0",  
    "STORE 190",  
    "PUSH 0",  
    "STORE 191",  
  
    "LOOP:",  
    "LOAD 191",  
    "LOAD 199",  
    "CMP",  
    "JZ END",  
    "LOAD 190",  
    "PUSH 200",  
    "LOAD 191",  
    "ADD",  
    "LOAD",  
    "ADD",  
    "STORE 190",  
    "LOAD 191",  
    "PUSH 1",  
    "ADD",  
    "STORE 191",  
    "JMP LOOP",  
    "END:",  
    "LOAD 190",  
    "HALT"  
]
```

Листинг 5 – Код запуска программы

```
def load_program():  
    code = assemble(program)  
    cpu.RAM[:len(code)] = code  
    cpu.RAM[200:210] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
    cpu.RAM[199] = 10  
load_program()  
update_gui()
```